

Name: _____





BOOTSTRAP

Equity • Scale • Rigor

Brought to you by the Bootstrap team:

- Emmanuel Schanzer
- Kathi Fiser
- Shriram Krishnamurthi
- Dorai Sitaram
- Joe Politz
- Ben Lerner
- Nancy Pfenning
- Flannery Denny
- Rachel Tabak

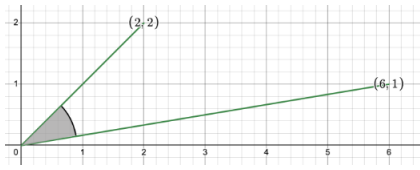
Measuring Similarity with AI

Another huge job AI does is **similarity**: given an input, find the things most *like* it. Every recommendation system — Spotify suggesting songs, Google finding matching images, a dating app proposing matches — is really a similarity engine.

The catch is that a computer doesn't *see* an image or *hear* a song. Where we see a mountain, the computer only sees a grid of **pixels** — numbers. So the first step is always the same:

Almost ANY kind of document can be described by a list of numeric attributes — and once it is, we can plot it as a *point in space*.

If every document is a point in space, then "similar" means "close together." But there's more than one way to measure "close"!

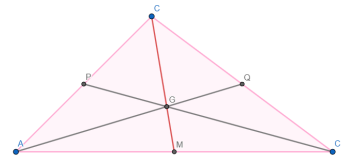
Simple (Exact-Match)	Distance	Angle
"Are the points identical?"	"How far apart are the points?"	"How aligned are the points?"
$(x_1, y_1) = (x_2, y_2)?$		
All-or-nothing: either two documents match precisely , or they don't.	Plot each document and measure the straight-line distance between points. Shorter Distance = More Similar.	Draw a line from the origin to each point and measure the angle difference between them. Smaller Angle = More Similar.

Converting documents into a table of points isn't *learning*: every document still has its own row, and a search just compares the query against *all* of them. That's a real and widely-used AI technique, but it's closer to a **lookup** than it is to **learning** — the computer is *memorizing* examples, not learning a general pattern from them.

Centroids: Supervised Learning

Labels unlock a powerful idea. If every image with a sun is a point in space, then "sun-ness" is just the point *in the middle* of those points.

This is the moment we finally **build a model** — and where real *learning* happens. A centroid throws away the individual points and keeps only their summary, **aggregating many documents into one and losing the original detail**. That is exactly what **training** means: a centroid *generalizes* a whole group into a single point, instead of memorizing every example.



Searching by nearest neighbor *memorizes* the data; building a centroid *generalizes* it into a model.

A centroid lets us reason about a whole **group** instead of a single document, which solves two big limitations of an unlabeled system.

- **Search by concept:** build the centroid of every song tagged "hip-hop," then find unlabeled songs close to it — so we can search by the *word* "hip-hop," not just by one example song.
- **Recommendation:** build a centroid for *liked* images, then find unlabeled images close to it — so we can search for recommendations based on all those songs, not just by one.

The Hidden Cost of Supervised Learning

Supervised learning only works because **humans** did the labeling — and that labor is often invisible. Every CAPTCHA you solve ("click all the traffic lights"), every paid data-labeler, and every content moderator is helping train someone's model. It's worth asking who does this work, and at what cost.

Plotting Images

For this page, you'll need [Image Similarity Starter File](#) open on your computer.

Plotting Images in 1 Dimension ("Green Space")

1) In the **Interactions Area**, evaluate `image-dot-plot(computed, "green", thumbnail)`, and hover your mouse over the images.

- What are the `green` values for our grassy images? grass and grass-sun. How far apart are they? _____
- How does this help us look for similarity? _____

2) Evaluate `distance-similarity(computed, "grass", [list: "green"])`. Was `"grass-sun"` the most similar? _____

Plotting Images in 2 Dimensions ("Blue-Green Space")

3) In the **Interactions Area**, evaluate `image-scatter-plot(computed, "green", "blue", thumbnail)`. Can we still calculate the distance? _____

4) Are the grassy images still clustered together? _____ Do you see any *other* clusters? _____

5) Will `distance-similarity` give us the right clusters for grass, snow, and sun in our 2-dimensional space?

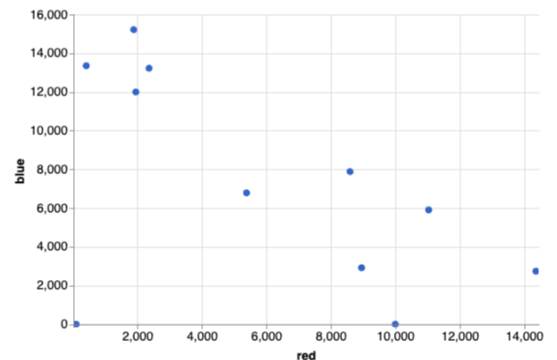
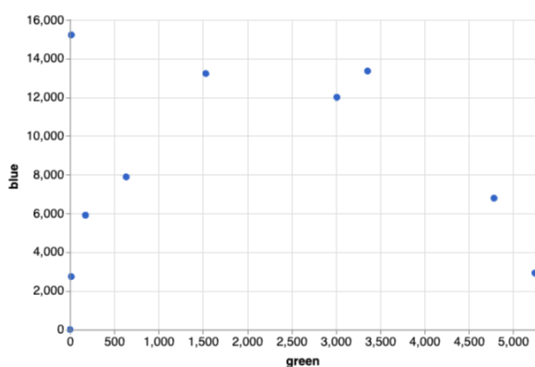
Check	Code	Clustered?
Grass	<code>distance-similarity(computed, "grass", [list: "green", "blue"])</code>	
Snow	<code>distance-similarity(computed, "snow2", [list: "green", "blue"])</code>	
Sun	<code>distance-similarity(computed, "sun1", [list: "green", "blue"])</code>	

6) What is the distance between the red squares (`red1` and `red2`) in our "Blue-Green Space"? _____

7) Looking at the raw data in the `computed` table, do you think `distance-similarity`, would you expect `red2` to be closer to `red1` or to `sunset` in other spaces? _____

Connecting Blue-Green and Blue-Red Space

8) The scatter plots below show our images plotted in Blue-v-Green and Blue-v-Red Space.

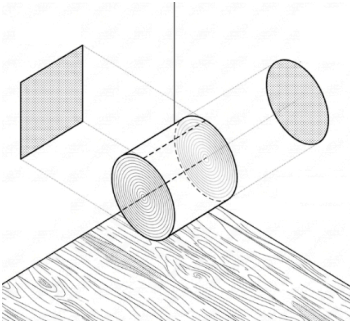


9) What's similar? _____

10) What's different? Why? _____

From Two Dimensions to Three

Projection



1) In your own words, explain what is happening in this picture. _____

Plotting Images in 2 Dimensions ("Blue-Green-Red Space")

You'll need the [RGB Image Plot \(Desmos\)](#). Desmos file open on your computer. At any time, you can click  to reset the file to the beginning.

- Our grass image is represented as a purple dot, while the points for our other images are shown in green.
- **Remember the mission!** We want to find the green point that's "most similar" to the purple one.

2) Which scatter plot from the bottom of [Plotting Images](#) matches what you're seeing in Desmos (circle one): Blue-Green Space Blue-Red Space

3) Turn on the "Blue-Red" folder, and explain what you see. _____

4) Click on any dot, drag it around to explore, and explain what you see. _____

5) Rotate the cube so that the Blue-Red scatter plot is at the "front" of the cube. Compare it to the Pyret scatter plot on the bottom of [Plotting Images](#).

6) Turn on the "Blue-Green-Red" folder, and rotate the cube as needed. Explain what you see. _____

7) Can you rotate the space so that the blue (3D) points *completely cover* the green (2D) ones? The red (2D) ones?

8) How is the 3D model in this desmos file like the image at the top of the page? _____

Lines and Angles

9) Turn on the "Lines" folder, then adjust the I_n slider draw lines to different Images. Explain what is happening. _____

10) Could you imagine two lines in this 3D space that start at the origin and **wouldn't** form an angle we can measure? If so, what are they? If

not, why not? _____

Angle Similarity (Unplugged)

Here's a simpler image model, with only a handful of rows representing small images, and only the columns for red, green, and blue.

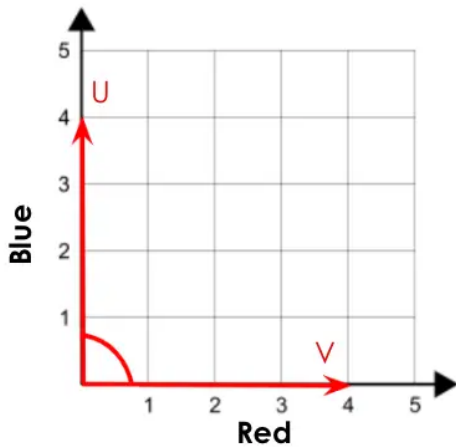
ID	red	green	blue
"U"	0	82	4
"V"	4	40	4
"W"	2	76	2
"X"	2	55	4
"Y"	1	98	1
"Z"	1	67	2

1) These images are *overwhelmingly* dominated by what color? _____ If we're looking to measure their differences, which colors make the most sense to use, and why?

To compare each pair of images in the questions below:

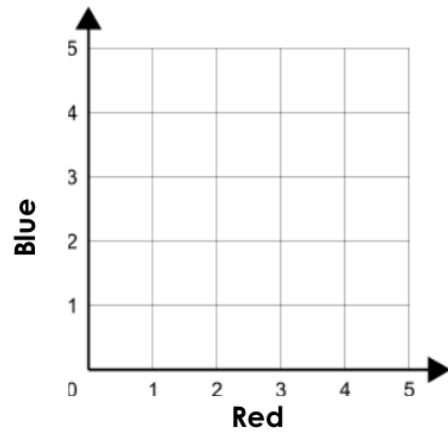
- Find the (red, blue) coordinates for each image in the table above.
- Plot the points on the graph provided, and draw a ray connecting the origin to each of the points.
- Use logic and your knowledge of angles to determine the *angle-similarity* of the 2 points.

2) Coordinate U: (0,4) Coordinate V: (4,0)



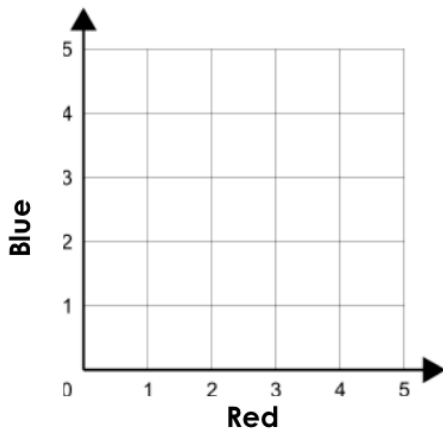
The angle-similarity between them is _____.

3) Coordinate Y: _____ Coordinate U: _____



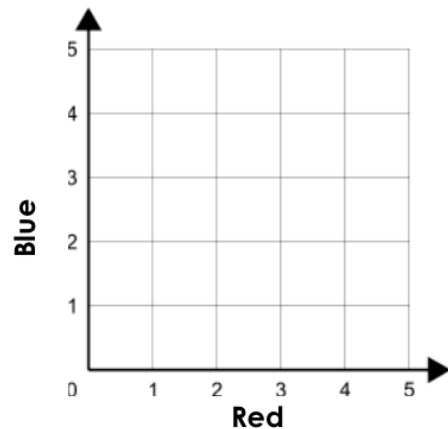
The angle-similarity between them is _____.

4) Coordinate W: _____ Coordinate V: _____



The angle-similarity between them is _____.

5) Coordinate X: _____ Coordinate Z: _____



The angle-similarity between them is _____.

Angle Similarity (Pyret)

Return to the [Image Similarity Starter File](#) and make sure you've clicked "Run".

Red v Blue

- Make `image-scatter-plot(computed, "red", "blue", thumbnail)`.
- Set the `x-min:` and `y-min:` to 0 and click "Redraw" so that we can imagine the rays that would connect our points to the origin.

1) Based on the image scatter plot, which image(s) do you expect to be most similar to "snow2" in the Blue-Red Space through the lens of `angle-similarity`? Why? _____

2) Compute `angle-similarity(computed, "grass", [list: "red", "blue"])`. Which Image does it compute to be the most similar? _____

Evaluate `angle-similarity(computed, "snow1", [list: "red"])`

3) Why are all of the measures for angle similarity zero? _____

Red v Green

- Make an image scatter plot comparing the Images based on red and green.
- Set the `x-min:` and `y-min:` to 0 and click "Redraw" so that we can imagine the rays that would connect our points to the origin.

4) Based on the image scatter plot, which restaurants do you expect to be most similar to the snowy image "snow1" in Red-Green space through the lens of `angle-similarity`? Why? _____

5) Compute `angle-similarity(computed, "grass", [list: "red", "green"])`. Which Image does it compute to be the most similar? _____

Entropy v Luminance

- Make an image scatter plot comparing the Images based on ENTROPY and LUMINANCE.
- Set the `x-min:` and `y-min:` to 0 and click "Redraw" so that we can imagine the rays that would connect our points to the origin.

6) Based on the image scatter plot, which restaurants do you expect to be most similar to the snowy image "snow1" in Entropy-Luminance space through the lens of `angle-similarity`? Why? _____

7) Compute `angle-similarity(computed, "snow1", [list: "ENTROPY", "LUMINANCE"])`. Which Image does it compute to be the most similar? _____

Red v. Green v. Blue v. LUMINANCE v. ENTROPY

8) What if we want to measure similarity, taking into account red v. green v. blue v. LUMINANCE, and ENTROPY? How can we use all five? _____

These materials were developed partly through support of the National Science Foundation (awards 1042210, 1535276, 1648684, and 1738598) and are licensed under a Creative Commons 4.0 Unported License. Based on a work at www.BootstrapWorld.org. Permissions beyond the scope of this license may be available by contacting contact@BootstrapWorld.org.